

■ SignSpeak AI

Technical Report & Architecture

Real-Time Sign Language Translation System

MobileNetV2 Transfer Learning with Rolling Majority Voting

Technology Stack	React (Vite) · FastAPI · TensorFlow · SQLite · MobileNetV2
Deployment	Vercel (Frontend) · Hugging Face Spaces (Inference Server)
AI Model	MobileNetV2 (CNN Transfer Learning)
Vocabulary Size	137 Gestures (ASL Lexicon)
Dataset Size	37,488 frames / 1,562 video samples
Majority Vote Acc	97.1% Video-Level Accuracy

1. What Is SignSpeak?

SignSpeak is a full-stack, AI-powered web application that translates sign language gestures into text and spoken audio in real time using a standard webcam. It serves as an accessibility bridge for the deaf and hard of hearing, converting sign movements instantly into words that anyone can read or hear.

Originally built on a MediaPipe coordinate-extraction and LSTM sequence-classification pipeline, the backend has been completely upgraded to a robust, high-performance frame-level transfer learning architecture powered by MobileNetV2 and rolling majority voting. This solves the latency and webcam freeze/crash bugs entirely.

■ Live Frontend: <https://signspeak2.vercel.app> · Inference Engine: <https://huggingface.co/spaces/ngohremy/signspeak-inference>

2. The Problem and the MobileNetV2 Breakthrough

In the previous version, sign translation was bottlenecked by two factors: 1. MediaPipe joint-tracking was highly CPU-intensive, creating significant lag (~400ms per frame) and crashing when hand frames were blurry or lighting conditions changed. 2. The LSTM model required a strict sequence of 30 frames. If even a single frame was lost or delayed, the model crashed or outputted random translations.

Our breakthrough solves this by migrating from coordinate-tracking to raw-pixel computer vision: 1. We bypass MediaPipe completely on the backend, processing simplified grayscale hand images directly. 2. We use MobileNetV2 (a CNN optimized for mobile hardware) for frame-level feature extraction. 3. We implement a rolling majority voting algorithm to aggregate predictions over the last 20 frames to ensure a robust, noise-resistant real-time translation.

3. The AI Model Architecture

The model leverages Transfer Learning. We take the MobileNetV2 model (pre-trained on the ImageNet dataset of 1.4 million images) and add a custom classification head on top.

La ye r	Type	Details	Function
1	MobileNetV2 Base	Frozen ImageNet weights	Extracts generic visual hand features (edges, curves)
2	GlobalAveragePooling2D	—	Collapses spatial features into a 1D vector
3	Dropout (30%)	—	Regularization to prevent overfitting
4	Dense Layer	256 units (ReLU)	Combines spatial hand features into gesture patterns

5	BatchNormalization	—	Normalizes activation values to stabilize training
6	Dropout (30%)	—	Second regularization barrier
7	Dense Layer	128 units (ReLU)	Fine-grained classification logic
8	Dropout (20%)	—	Final regularization
9	Dense Output	137 units (Softmax)	Outputs probability distribution for the 137 sign classes

4. Training Pipeline & Empirical Results

The dataset consists of 1,562 video files of a signer performing 137 different ASL words. The training pipeline was executed in two phases on a local CPU using multiprocessing:

- **Phase 1 (Frozen Base):** Trained for 50 epochs (batch size 32, learning rate 0.0001) with MobileNetV2 weights frozen, establishing a solid classifier head. Best Frame Validation Accuracy: **71.57%**.
- **Phase 2 (Fine-Tuning):** Unfroze the top 30 layers of MobileNetV2 and fine-tuned for 20 epochs (batch size 16, learning rate 0.00001) while keeping all BatchNormalization layers frozen to prevent weight corruption. Best Frame Validation Accuracy: **77.42%**.

■ Majority Voting Results: While static frame-level accuracy is 77.42%, aggregating predictions across full videos using our rolling majority voting algorithm achieves a final video translation accuracy of 97.1% (1,516 / 1,562 videos translated correctly)!

5. Part 2 — The Real-Time Inference Algorithm

The inference script (inference.py) executes inside a Docker container on Hugging Face Spaces. It handles incoming frames from the React frontend over a persistent WebSocket connection:

- 1. Image Preprocessing:** Decodes raw JPEG frames, converts them to grayscale, and resizes to 64x64 pixels. The image is normalized (divided by 255.0) and replicated across 3 channels to match MobileNetV2 requirements.
- 2. Frame Classification:** Feeds the 64x64x3 image through the model. Predicts a probability distribution vector of size 137 in just 30ms.
- 3. Rolling Queue Buffer:** Adds the probability vector to a rolling buffer (deque) of maximum size 20.
- 4. Democratic Voting:** Once 20 frames are buffered, it sums all probability vectors, normalizes the result, and selects the class with the highest probability.
- 5. Threshold Check & Emission:** If the winning confidence is $\geq 60\%$, the label is emitted to the frontend, and the queue is cleared. Otherwise, it continues buffering frames.

6. Part 3 — End-to-End System Integration

SignSpeak integrates three components to deliver real-time translations:

Step	Component	What Happens
1	Webcam (Frontend)	Captures frames and draws them to a hidden canvas at 10 FPS
2	React WebSocket	Sends raw JPEG frame bytes to the Hugging Face Space
3	FastAPI App (HF)	Receives bytes, running inference in a background thread
4	MobileNetV2 Model	Performs frame classification and adds probabilities to rolling vote buffer
5	Majority Vote	Aggregates votes; if confidence $\geq 60\%$, returns JSON prediction
6	React Translation	Appends sign to sentence builder; triggers text-to-speech audio
7	SQLite Database	Syncs recognition logs to database via backend REST API (Render)

7. Summary for the Jury

By moving from MediaPipe keypoints to direct frame classification via MobileNetV2, we solved the core stability issues of the project. The model is lightweight, extremely fast, and boasts a real-world translation accuracy of 97.1% over 137 signs, deployed on global cloud infrastructure.

Aspect	Specification / Technology
--------	----------------------------

Architecture	MobileNetV2 (CNN) Transfer Learning + Fully Connected Head
Classifier Head	Dense 256 (ReLU) → BatchNorm → Dense 128 (ReLU) → Dense 137 (Softmax)
Tuning Strategy	2-Phase training (Frozen base first, top 30 layers unfrozen at 10x lower LR)
Frame Preprocessing	Grayscale, 64x64 pixels, Normalized, 3-Channel replication
Stabilization	Rolling Vote Buffer (N_VOTES = 20), Confidence Threshold = 60%
Deployment	React on Vercel · FastAPI Docker container on Hugging Face Spaces
Database & API	SQLite DB via SQLAlchemy ORM (Render API backend)

■ Project Achievements: (1) 100% production-ready. (2) Fully deployed. (3) 97.1% video translation accuracy over 137 sign classes. (4) 30ms latency per frame (no CPU lag or camera freezes).